

HO CHI MINH CITY UNIVERSITY OF TRANSPORT
FACULTY OF INFORMATION TECHNOLOGY
SOFTWARE ENGINEERING DEPARTMENT

CHAPTER 9
STRINGS AND LISTS

CONTENTS

1. Strings

2. Lists

1. Strings

- A string is a sequence of characters. Python treats characters and strings the same way.
- Python has a number of powerful features for manipulating strings:
  - Creating Strings
  - String Indexing
  - String Slicing
  - Modifying Strings
  - Iterating a String
  - String Operators
  - String Functions

Creating Strings

- **Creating strings** as follows:

```
1  s1 = str()     # Create an empty string object
2  s2 = str("Welcome") # Create a string object for Welcome
```

- Or

```
1  s1 = ""# Same as s1 = str()
2  s2  = "Welcome"# Same as s2 = str("Welcome")
```

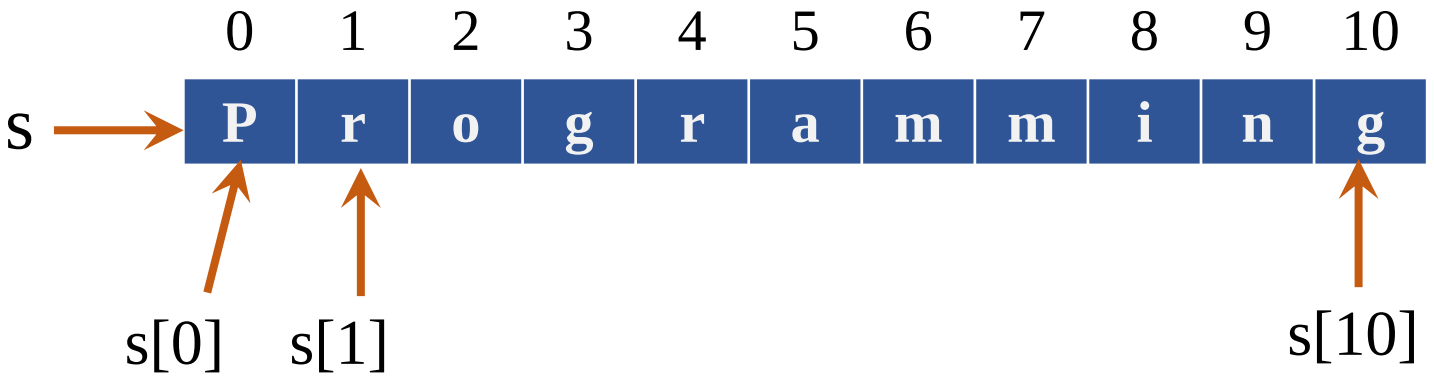
- **A string object is immutable**: once it is created, its content cannot be changed.

String Indexing

- A character in the string can be accessed through the index operator using the syntax:

  s[<index>]

- For example: s = “Programming”



String Indexing

- The indexes are 0 based:

  indexes range from 0 to len(s) – 1

```
1  s = "Programming"
2  print(s[0])
3  print(s[1])
4  print(len(s))
5  print(s[len(s)-1])
6  #print(s[len(s)]) --> IndexError: string index out of range
```

```
P
r
11
g
```

String Indexing

- Python also allows the use of negative numbers as indexes → indexing occurs from the end of the string backward:

-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
P	**r**	**o**	**g**	**r**	**a**	**m**	**m**	**i**	**n**	**g**
0	1	2	3	4	5	6	7	8	9	10

```
1  s = "Programming"
2  print(s[-len(s)])
3  print(s[-1])
```

```
P
g
```

- **The actual position is obtained by adding the length of the string with the negative index.**

String Slicing

- The slicing operator returns a slice of the string using the syntax:

s[start : end : step]

- The slice is a substring from index **start** to index **end – 1.**
- The *starting index or ending index may be omitted*. In this case, by default the starting index is 0 and the ending index is the last index

```
1  s = "Programming"
2  print(s[1:4])
3  print(s[:4])
4  print(s[4:])
5  print(s[1:-1])
6  print(s[3:1]) # the slice is empty
```

```
rog
Prog
ramming
rogrammin
```

Modifying Strings

- Can we change individual characters of a string?
- For example:

```
1  s = "Programming"
2  s[3] = "X"
```

→ Error

- **Solution**: build a new string from s and reassign it to s

```
1  s = "Programming"
2  s = s[:3] + "X" + s[4:]
3  print(s)
```

```
ProXramming
```

Iterating a String

- A string is iterable → can use a for loop to traverse all characters in the string sequentially.

```
1  s = "Programming"
2  for ch in s:
3      print(ch,end='-')
```

```
P-r-o-g-r-a-m-m-i-n-g-
```

```
1  # displays characters at odd-numbered positions
2  s = "Programming"
3  for i in range(1,len(s),2):
4      print(s[i],end='-')
```

```
r-g-a-m-n-
```

String Operators

- **Concatenation (+):** combines two strings
- **Repetition (*) :** repeats a string a certain number of times
- **Substring Testing** (**in** and **not in**): to test whether a string is in another string

```
1  s1 = "Welcome"
2  s2 = "Python"
3  s = s1 + " to " + s2
4  print(s)
```

```
Welcome to Python
```

```
1  s = input("Enter a string:")
2  if "Python" in s:
3      print("Python is in",s)
4  else:
5      print("Python is in",s)
```

```
Enter a string:Python is fun
Python is in Python is fun
```

String Operators

- **Comparing Strings**
  - Using the comparison operators (==,!=, >, >=, <,<=)
  - Python compares strings by comparing their corresponding characters, and it does this by evaluating the characters' numeric codes

```
1  s1 = "apple"
2  s2 = "avocado"
3  print(s1 == s2)
4  print(s1 < s2)
5  print(s1 <= s2)
```

```
False
True
True
```

```
1  "ab" > "abc"
```

```
False
```

String Functions

Built-in String Functions

- **ord(ch)**: returning the ASCII code for the character ch.
- **chr(code)**: returning the character represented by the code

```
1  print(ord('A')) # --> 65
2  print(chr(98)) # --> 'b'
3
4  offset = ord('a') - ord('A')
5  print(offset) # --> 32
6  lower = 'z'
7  upper = chr(ord(lower) - offset)
8  print(upper) # --> 'Z'
9
```

```
65
b
32
Z
```

String Functions

Case Conversion

- **s.lower()**: return a copy of string in lowercase
- **s.upper():** return a copy of string in uppercase
- **s.capitalize():** returns a copy of string with the first character converted to uppercase.
- **s.title():** returns a copy of this string with the first letter capitalized in each word.
- **s.swapcase():** Returns a copy of this string in which lowercase letters are converted to uppercase and uppercase to lowercase

```
1  s1 = "Welcome to Programming"
2  print(s1.lower())
3  print(s1.upper())
4  print(s1.swapcase())
5
```

```
welcome to programming
WELCOME TO PROGRAMMING
wELCOME TO pROGRAMMING
```

String Functions

Stripping characters from a String

- **s.strip([<char>])**: removes any character from the beginning or the end
  - By default, char is **None** --> ***whitespace characters are removed***

```
1  s = " Welcome to Python \n"
2  s.strip()
```

```
'Welcome to Python'
```

```
1  s = "***10***"
2  s.strip('*')
```

```
'10'
```

String Functions

Find and Replace

- **s.count(sub[,start[,end]])**: Returns the number of non-overlapping occurrences of this substring
- **s.endswith(suffix[,start[,end]])**: Returns **True** if the string ends with the specified suffix.
- **s.startswith(prefix[,start[,end]])**: Returns **True** if the string starts with the specified prefix.

```
1  s = "Welcome to Python"
2  print(s.count("o"))
3  print(s.count("o",7))
```

```
3
2
```

```
1  print(s.endswith("thon"))
2  print(s.startswith("good"))
```

```
True
False
```

String Functions

Find and Replace

- **s.find(sub[,start[,end]]):** returns the lowest index in s where substring is found, otherwise return -1
- **s.replace(old, new):** Returns a new string that replaces all the occurrences of the old string with a new string

```
1  print(s.find("come"))
2  print(s.find("become"))
```

```
3
-1
```

```
1  s1 = s.replace("Python","Java")
2  print(s1)
```

```
Welcome to Java
```

String Functions

Character Classification

- **s.isalnum()**: returns **True** if s is non-empty and all its characters are alphanumeric (either a letter or a number)
- **s.isalpha()**: returns **True** if s is non-empty and all its characters are alphabetic
- **s.isdigit()**: returns **True** if s is non-empty and all its characters are numeric digits

```
1  s = "python3"
2  print(s.isalnum())
```

```
True
```

```
1  s = "abc"
2  print(s.isalpha())
```

```
True
```

```
1  s = "2019"
2  print(s.isdigit())
```

```
True
```

String Functions

Character Classification

- **s.islower()**: returns **True** if s is non-empty and all the alphabetic characters it contains are lowercase.
- **s.isupper()**: returns **True** if s is non-empty and all the alphabetic characters it contains are uppercase

```
1  s = "Hello"
2  print(s.islower())
3  print(s.isupper())
```

```
False
False
```

String Functions

Split

- **s.split(sep=None, maxsplit=-1)**: Splits a string into a list of substrings.
  - **Sep**: delimiter according which to split the string, default value is **None** (whitespace)
  - **Maxsplit**: maximum number of splits to do, default value is -1 (no limit)

```
1  s = "This is split function"
2  items = s.split()
3  print(items)
```

```
['This', 'is', 'split', 'function']
```

```
1  date = "20/11/2011"
2  items_1 = date.split(sep='/')
3  print(items_1)
4
5  items_2 = date.split(sep='/',maxsplit=1)
6  print(items_2)
```

```
['20', '11', '2011']
['20', '11/2011']
```

2. Lists

- Suppose that you need to read 100 numbers, compute their average, and then find out how many of the numbers are above the average.
- How do you solve this problem?
- Create 100 variables??? → This way is impractical.

Lists

Lots of data

How can we store this much data in Python?

We would need 5x19 variables...

→The solution is to store all values together in a **list.**

#	Country	G	S	B	Total
1	Norway	14	14	11	39
2	Germany	14	10	7	31
3	Canada	11	8	10	29
4	United States	9	8	6	23
5	Netherlands	8	6	6	20
6	Sweden	7	6	1	14
7	South Korea	5	8	4	17
8	Switzerland	5	6	4	15
9	France	5	4	6	15
10	Austria	5	3	6	14
11	Japan	4	5	4	13
12	Italy	3	2	5	10
13	Olympic Athletes from Russia	2	6	9	17
14	Czech Republic	2	2	3	7
15	Belarus	2	1	0	3
16	China	1	6	2	9
17	Slovakia	1	2	0	3
18	Finland	1	1	4	6
19	Great Britain	1	0	4	5

Lists

- **A list is a ordered sequence of values**. The values that make up a list are called its **elements**, or its **items**.
- List is a collection data type.
- The elements of a list can be any type.
- Common operations for manipulating lists:
  - Creating Lists
  - List Indexing
  - List Slicing
  - Modifying List
  - Traversing a List
  - List Operators
  - List Functions
  - List Comprehensions
  - Copying Lists

Creating Lists

- The elements in a list are separated by commas and are enclosed by a pair of brackets []
- A list can contain the elements of the same type or mixed types.

```
1  empty_list = [] # empty list
2  print(empty_list)
3  gold = [14,14,11]
4  print(gold)
5  countries = ["Norway","Germany","Canada"]
6  print(countries)
```

```
[]
[14, 14, 11]
['Norway', 'Germany', 'Canada']
```

Creating Lists

```
1  #mix of different types
2  mix_list = [2,"three",4]
3  print(mix_list)
4
5  #make a list of lists
6  hands = [['J','Q','K'],['2','2','2'],['6','A','K']]
7  print(hands)
```

```
[2, 'three', 4]
[['J', 'Q', 'K'], ['2', '2', '2'], ['6', 'A', 'K']]
```

```
1  # Create a list that consists integer 0 -> n-1
2  n = int(input("n = "))
3  mylist = list(range(n))
4  print(mylist)
```

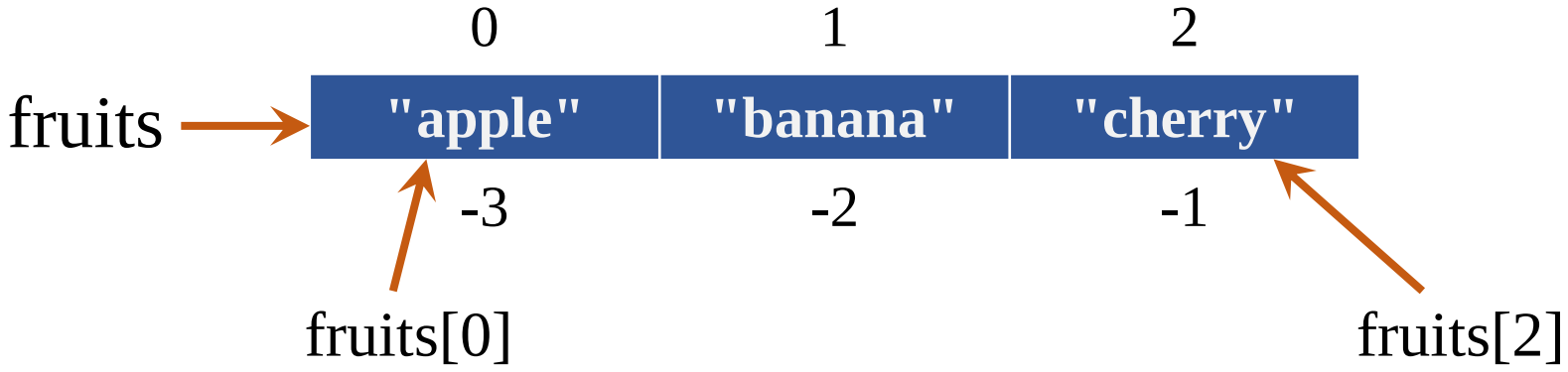
```
n = 10
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

List Indexing

- We can access the elements of a list using an integer index.

myList[index]

- For example, fruits= ["apple", "banana", "cherry"]




```
1  fruits= ["apple","banana","cherry"]
2  print(fruits[0])
3  print(fruits[-1])
```

```
apple
cherry
```

List Indexing

- A list can contain sublists.
- To access the elements in a sublist, simply append an additional index:

```
1  hands = [['J','Q','K'],['2','2','2'],['6','A','K']]
2  print(hands[0])
3  print(hands[0][1])
4  print(hands[2][1])
```

```
['J', 'Q', 'K']
Q
A
```

List Slicing

- The slicing operator returns a slice of the list using the syntax:

  mylist[start : end]

- The slice is a sublist from index **start** to index **end – 1**

```
1  primes = [2, 3, 5, 7,11,13]
2  print(primes[2 : 4])
3  print(primes[:2]) # two first primes
4  print(primes[3:]) # all the primes from index 3
5  print(primes[:]) # all the primes
6  print(primes[1:-1]) # All the primes except the first and last
```

```
[5, 7]
[2, 3]
[7, 11, 13]
[2, 3, 5, 7, 11, 13]
[3, 5, 7, 11]
```

Modifying List

- Lists are "mutable", meaning we can change their elements.
- To change the value of a specific element, refer to the index number:

```
1 fruits= ["apple","banana","cherry"]
2 fruits[1] = "grapes"
3 print(fruits)
```

```
['apple', 'grapes', 'cherry']
```

```
1 fruits[1:] = ["kiwi","durian"]
2 print(fruits)
```

```
['apple', 'kiwi', 'durian']
```

```
1 fruits[1:] = [] # remove elements from 1 to last
2 print(fruits)
```

```
['apple']
```

Traversing a List

- Using **for** loop, which enables you to traverse the list sequentially without using an index variable.

```
1 fruits= ["apple","banana","cherry"]
2 for f in fruits:
3     print(f,end=" ")
```

```
apple banana cherry
```

- Use an index variable if you wish to traverse the list in a different order or change the elements in the list.

```
1 fruits= ["apple","banana","cherry"]
2 size = len(fruits)
3 for i in range(size-1, -1, -1): # i = size-1 --> 0
4     print(fruits[i],end=" ")
```

```
cherry banana apple
```

Traversing a List

```
1  numbers = [4,2,8,7]
2  s = 0
3  for num in numbers:
4      s = s + num
5  print("Sum all elements in list: ",s)
```

```
Sum all elements in list:  21
```

```
1  # Print all positive elements in list
2  numbers = eval(input("Enter a list of integer: "))
3  flag = False
4  for num in numbers:
5      if num > 0:
6          flag = True
7          print(num)
8  if not flag:
9      print("The list has no positive elements!")
```

```
Enter a list of integer: [-9,0,4,-2,5]
4
5
```

List Operators

Operator	Description
in	True if element x is in list.
not in	True if element x is not in list.
+	Concatenates list1 and list2.
*	n copies of list concatenated.

```
1  countries = ["Norway","Germany","Canada"]
2  "Canada" in countries
```

```
True
```

```
1  list1 = [1,2,3]
2  list2 = [4,5,6]
3  list3 = list1 + list2
4  print(list3)
5  print(list1*2)
```

```
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 1, 2, 3]
```

List Operators

- **Comparing Lists**
  - Using the comparison operators (>, >=, <, <=, ==,!=)
  - The comparison uses ordering: the first two elements are compared, and if they differ this determines the outcome of the comparison; if they are equal, the next two elements are compared, and so on.

```
1  color1 = ["green", "red", "blue"]
2  color2 = ["red", "blue", "green"]
3  list1 = [1,2,3]
4  print(color1 != color2)
5  print(color1 > color2)
6  #color1 > list1 --> Error
```

```
True
False
```

List Functions

- **mylist.append(<object>)**: appends object to the end of list

```
1  mylist = [5, 3, 4, 1, 6, 4]
2  mylist.append(10)
3  print(mylist)
```

```
[5, 3, 4, 1, 6, 4, 10]
```

```
1  # Create a list with n elements
2  n = int(input("Enter number of elements in list: "))
3  mylist = []
4  for i in range(n):
5      value = eval(input(f"mylist[{i}] = "))
6      mylist.append(value)
7  print("My list is", mylist)
```

```
Enter number of elements in list: 3
mylist[0] = 6
mylist[1] = 1
mylist[2] = 2
My list is [6, 1, 2]
```

List Functions

- **mylist.extend(<iterable>)**: Extend list by appending elements from the iterable, behaves like the + operator.

```
1  list1 = [2,3,4]
2  list1.extend([5,6])
3  print(list1)
4  list1.extend("Python")
5  print(list1)
```

```
[2, 3, 4, 5, 6]
[2, 3, 4, 5, 6, 'P', 'y', 't', 'h', 'o', 'n']
```

- **mylist.count(<object>)**: Returns the number of times element <object> appears in the list

```
1  list1 = [3,1,4,5,4]
2  list1.count(4)
```

```
2
```

List Functions

- **mylist.index(<object>)**: Returns the index of the first occurrence of element <object> in the list.

```
1  list1 = [3,4,1,5,4]
2  list1.index(4)
```

```
1
```

- **mylist.insert(<index>, <object>)**: Inserts an element <object> at a given index.

```
1  list1 = [1,5,7,9]
2  list1.insert(1,3)
3  print(list1)
```

```
[1, 3, 5, 7, 9]
```

List Functions

- **mylist.remove(<object>)**: Removes the first occurrence of element x from the list.

```
1  list1 = [3,4,1,5,4]
2  list1.remove(4)
3  print(list1)
4  #list1.remove(6) --> Error: 6 not in list
```

```
[3, 1, 5, 4]
```

- **mylist.pop(index = -1)**: Removes the element at the given position and returns it.

```
1  list1 = [3,4,1,5,4]
2  list1.pop(1) # return 4
3  print(list1)
4  list1.pop()
5  print(list1)
```

```
[3, 1, 5, 4]
[3, 1, 5]
```

List Functions

- **mylist.reverse()**: Reverses the elements in the list.

```
1  odd = [1,3,5,7,9]
2  odd.reverse()
3  print(odd)
```

```
[9, 7, 5, 3, 1]
```

- **mylist.sort()**: Sorts the elements in the list in ascending order.

```
1  odd = [5,1,7,3,9]
2  odd.sort()
3  print(odd)
```

```
[1, 3, 5, 7, 9]
```

List Comprehensions

- Provide a concise way to create a sequential list of elements.

```
1  squares = []
2  for i in range(5):
3      squares.append(i*i)
4  print(squares)
```

```
[0, 1, 4, 9, 16]
```



```
1  squares = [i*i for i in range(5)]
2  print(squares)
```

```
[0, 1, 4, 9, 16]
```

- Syntax:

new_list = [expression for member in iterable (if conditional)]

  - A list comprehension consists of brackets containing an expression followed by a **for** clause, then zero or more **for** or **if** clauses.

List Comprehensions

Example 1

```
1 numbers = [4,1,3,5,2,6]
2 even = [x for x in numbers if x%2 == 0]
3 print(even)
```

```
[4, 2, 6]
```

Example 2

```
1 fruits = ["apple","avocado","banana","cherry","watermelon"]
2 short_fruits = [f for f in fruits if len(f) <= 6]
3 print(short_fruits)
```

```
['apple', 'banana', 'cherry']
```

Example 3

```
1 fruits = ["apple","avocado","banana","cherry","watermelon"]
2 short_fruits = [f.upper() for f in fruits if len(f) <= 6]
3 print(short_fruits)
```

```
['APPLE', 'BANANA', 'CHERRY']
```

Copying Lists

- Using the assignment statement (=) to copy the data in one list to another list → **Result???**

```
1  list1 = [1,3,5,7]
2  list2 = list1
3  print("Before appending:",end=" ")
4  print(len(list1))
5  list2.append(9)
6  print("After appending:",end=" ")
7  print(len(list1))
8  print("After modifying")
9  list1[2] = 11
10 print(list2)
```

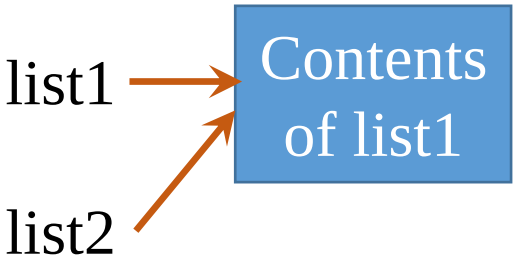
```
Before appending: 4
After appending: 5
After modifying
[1, 3, 11, 7, 9]
```

```
1  list1 = [1,3,5,7]
2  list2 = [1,3,5,7]
3  print("Before appending:",end=" ")
4  print(len(list1))
5  list2.append(9)
6  print("After appending:",end=" ")
7  print(len(list1))
8  print("After modifying")
9  list1[2] = 11
10 print(list2)
```

```
Before appending: 4
After appending: 4
After modifying
[1, 3, 5, 7, 9]
```

Copying Lists

- list2 = list1 → Do not copy the contents
- So, get a duplicate copy of list1 into list2,



you have to copy individual elements from the source list to the target list.

```
1  list1 = [1,3,5,7]
2
3  list2 = [x for x in list1]
4
5  print(list2)
6  print(list2 is list1)
```

```
[1, 3, 5, 7]
False
```

Or simply

```
1  list1 = [1,3,5,7]
2
3  list2 = [] + list1
4
5  print(list2)
6  print(list2 is list1)
```

```
[1, 3, 5, 7]
False
```